Digital Systems

Lecture 6

Subroutines

Mark van der Wilk

Hilary Term 2025

Department of COMPUTER SCIENCE

UNIVERSITY OF OXFORD

Following the course of past years by Mike Spivey

Previously

We wrote a single subroutine, but with limitations:

-
-

Previously

We wrote a single subroutine, but with limitations:

- Could not call other subroutines.
-

Previously

We wrote a single subroutine, but with limitations:

- Could not call other subroutines.
- Could only use `r0` - `r3`, while leaving `r4` and above unchanged.

Previously

We wrote a single subroutine, but with limitations:

- Could not call other subroutines.
- Could only use `r0` - `r3`, while leaving `r4` and above unchanged.

These kinds of subroutines are very useful, because they are very **efficient**.

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

How can we accept more than 4 arguments?

- Arguments are stored in `r0` - `r3`. Where do additional ones go?

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

How can we accept more than 4 arguments?

- Arguments are stored in `r0` - `r3`. Where do additional ones go?

What if we need more local memory than the registers?

- Where should we store these?

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

How can we accept more than 4 arguments?

- Arguments are stored in `r0` - `r3`. Where do additional ones go?

What if we need more local memory than the registers?

- Where should we store these?

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

How can we accept more than 4 arguments?

- Arguments are stored in `r0` - `r3`. Where do additional ones go?

What if we need more local memory than the registers?

- Where should we store these?

Today we will discuss how to use RAM to solve these problems.

Subroutines should be black-box

Subroutines should work no matter who calls them. Instructions inside are fixed!

- We need conventions that allow the same instructions to perform the same operation, **regardless of the state of the calling subroutine(s)**.

Subroutines should be black-box

Subroutines should work no matter who calls them. Instructions inside are fixed!

- We need conventions that allow the same instructions to perform the same operation, **regardless of the state of the calling subroutine(s)**.

Store registers in fixed memory addresses?

Subroutines should be black-box

Subroutines should work no matter who calls them. Instructions inside are fixed!

- We need conventions that allow the same instructions to perform the same operation, **regardless of the state of the calling subroutine(s)**.

Store registers in fixed memory addresses?

We cannot use absolute memory addresses

What about subroutines calling themselves (indirectly)?

Use a *Stack* for local variables

-
-
-

Use a *Stack* for local variables

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
-
-

Use a *Stack* for local variables

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
-

Use a *Stack* for local variables

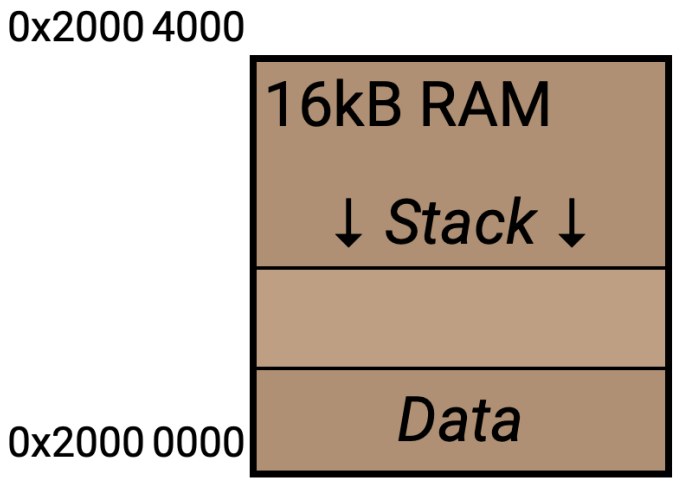
- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
- Memory above in the stack is free to be used by the subroutine.

Use a *Stack* for local variables

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
- Memory above in the stack is free to be used by the subroutine.

This is only one possible solution, but is the one chosen by design in ARM. Special instructions are created to facilitate using the stack.

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

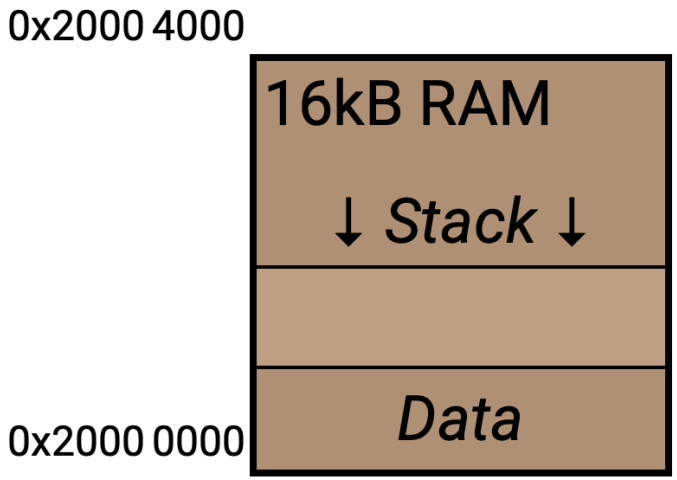
`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

-
-
-

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

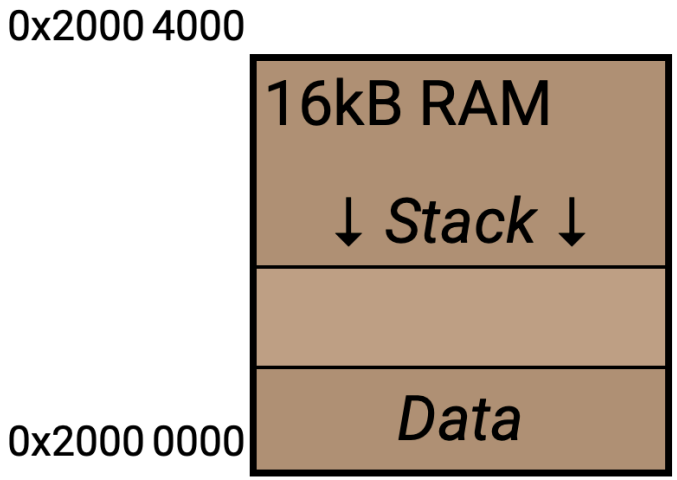
`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So “above” in the stack has a lower address!
-
-

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

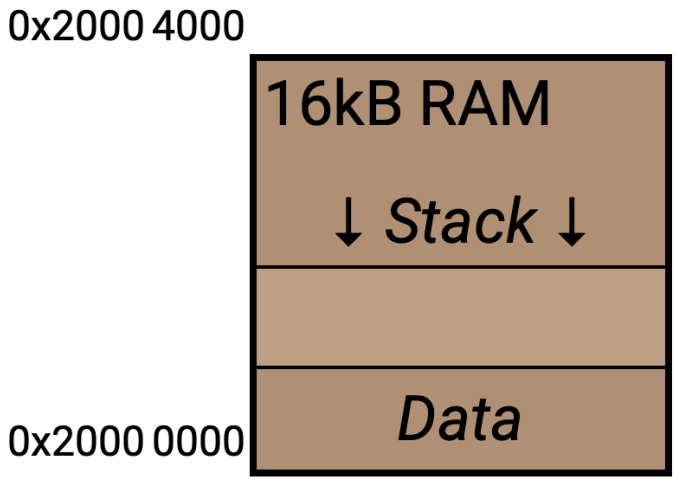
`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So "above" in the stack has a lower address!
- Each time a subroutine is called, `sp` points to a usable region of memory (stack frame), that won't interfere with other subroutines.
-

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So "above" in the stack has a lower address!
- Each time a subroutine is called, `sp` points to a usable region of memory (stack frame), that won't interfere with other subroutines.
- When a subroutine returns, the caller finds its stack frame as it left it.

Subroutine Contract

Calling function:

- Places first arguments 1 … 4 in `r0` - `r3`
- Places arguments ≥ 4 on stack (called function knows how many to expect) (labs won't need this)

Subroutine Contract

Calling function:

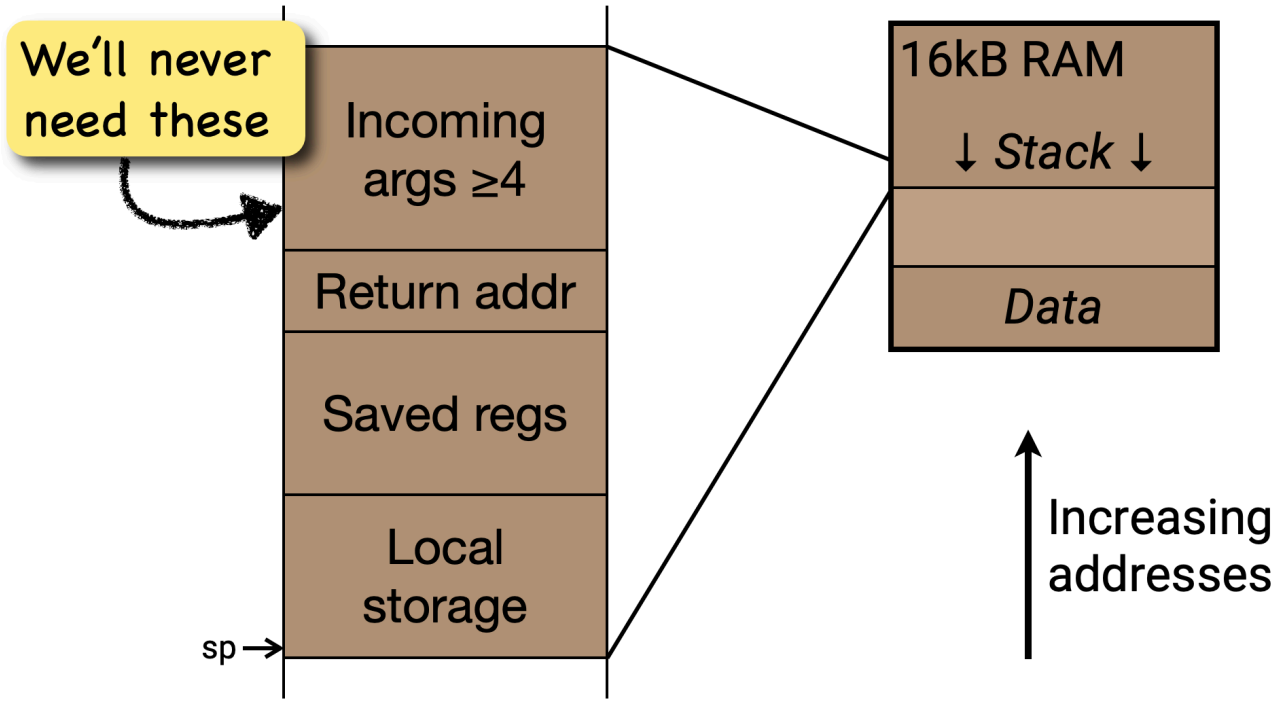
- Places first arguments 1 … 4 in `r0` - `r3`
- Places arguments ≥ 4 on stack (called function knows how many to expect) (labs won't need this)

Called function:

- Leave values of `r4` and up unchanged on return
- Leave stack pointer `sp` unchanged on return
- Leave stack below current location (i.e. memory addresses above `sp`) unchanged
- Leave return value in `r0`

Subroutine Convention

Subroutine can use stack in whatever way it wants (assuming there is space!), but ARM provides instructions for:



Storing Registers

ARM provides two assembly instructions to make restoring state following subroutines easy:

-

-

Storing Registers

ARM provides two assembly instructions to make restoring state following subroutines easy:

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
-

Storing Registers

ARM provides two assembly instructions to make restoring state following subroutines easy:

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
- `pop {REGISTER_LIST}`: Take values from stack, and place into the registers in list order, and increase `sp` accordingly.

Storing Registers

ARM provides two assembly instructions to make restoring state following subroutines easy:

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
- `pop {REGISTER_LIST}`: Take values from stack, and place into the registers in list order, and increase `sp` accordingly.

```
func:
    push {r4, r5, lr} @ Save registers
    @ Use r4 and r5, call other routines
    pop {r4, r5, pc} @ Restore and return
```

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0` - `r7`)
-
-

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
-

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Prev, we saw we could return from a subroutine using `bx lr`. Using `pop` to restore to `pc` gives the same effect, but without first restoring to `lr`!

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Prev, we saw we could return from a subroutine using `bx lr`. Using `pop` to restore to `pc` gives the same effect, but without first restoring to `lr`!

`push`/`pop` depart from RISC principles!

Normally, one instruction does one thing, but this gives speedup! Load/store operation is 2 cycles, while `push`/`pop` is 1+n cycles.

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Prev, we saw we could return from a subroutine using `bx lr`. Using `pop` to restore to `pc` gives the same effect, but without first restoring to `lr`!

> **`push`/`pop` depart from RISC principles!**
>
> Normally, one instruction does one thing, but this gives speedup! Load/store operation is 2 cycles, while `push`/`pop` is 1+n cycles.

Remember, instruction set designers optimised the common case.

Putting it all together

Example: Factorial

```
int fac(int n) {
    int k = n, f = 1;
    while (k != 0) {
        f = mult(f, k);
        k = k-1;
    }
    return f;
}
```

Subroutine calls another subroutine! So we will use what we discussed.

Example: Factorial, opening

- As usual, let's start by converting code to assembly, and not worrying about efficiency.
- For clarity, I recommend making clear which variables are assigned to which registers (little table in your exam answer helps!)
- Let's assign `k` to `r4`, and `f` to `r5` (logic behind this?)
- Must use `push`.

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
```

Example: Factorial, Loop

Again, we can worry about optimisations later:

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished
```

Example: Factorial, Subroutine Call

Again, we can worry about optimisations later:

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

Example: Factorial, Finish Loop

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0

    subs r4, r4, #1             @ Decrement k
    b again                     @ And repeat
```

Example: Factorial, Return Subroutine

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

...

finish:
    movs r0, r5                 @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, All Together

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

```
    subs r4, r4, #1          @ Decrement k
    b again                  @ And repeat
finish:
    movs r0, r5              @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, optimisation

You may have spotted that `f` can live in `r0` constantly, since the return value of `mult()` will also be the first argument for the next call.

Optimising compilers can spot this.

Summary

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

Summary

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

Summary

How can we create subroutines which can call others?

- Remember value of `lr` that will get overwritten by next jump.

How can we use `r4` and above?

- Must restore of `r4` and above at the end of subroutine.

- See notes for:
  - Expanded discussion
  - More efficient multiplication routine
- Don't forget rainbow chart and instruction overview!